Lincoln University College

Faculty of Computer Science and Multimedia

CafeFlow - A Cafe Management System

A PROJECT REPORT

Submitted to

Department of Information Technology

Phoenix College of Management

Maitidevi, Kathmandu

In partial fulfillment of the requirements for the Bachelor in Information Technology (BIT)

Submitted by

Amit Tharu

BIT 7th Semester

University ID: LC0003001642

2026/05/30

DECLARATION

I hereby declare that the project report entitled **"CafeFlow - A Cafe Management System"** is my original work, completed under the guidance of **Mr. Saishab Bhattarai**. This report has not been submitted elsewhere, in full or in part, for the award of any degree, diploma, or other academic recognition.

Signature:
Name: Amit Tharu
Date: 2026/05/30

LETTER OF APPROVAL

Lincoln University College

Faculty of Computer Science and Multimedia

Phoenix College of Management

Maitidevi, Kathmandu

Bachelors in Information Technology (BIT)

The Project report entitled **"CafeFlow - A Cafe Management System"** submitted by **Amit Tharu**, in partial fulfillment of the requirements for the Bachelor's degree in Information Technology, has been accepted as a record of work independently carried out by the student.

In our opinion, it is satisfactory in the scope and quality of a project for the required degree.

Er. Santosh Dhungana
Head of Department

Mr. Bijay Mishra
Program Coordinator

Mr. Saishab Bhattarai
Internal Examiner

External Examiner

ABSTRACT

CafeFlow is a web-based cafe management system designed to digitise the daily operations of small and medium cafes. Many cafes still rely on manual order taking, paper bills, and spreadsheet-based stock tracking, which leads to slow service, billing errors, stock-outs, and a lack of reliable sales insight. CafeFlow brings menu management, order taking, billing, inventory tracking, staff management, and sales analytics together in a single role-based platform. Staff can build and submit orders, advance them through a pending–preparing–completed workflow, and generate itemised bills, while administrators additionally manage the menu, monitor stock and low-stock alerts, manage the team, and view revenue and best-seller analytics derived from real order data. The system is built with Next.js, React, and TypeScript on top of a SQLite database, with secure email/password authentication and role-based access control. By replacing manual processes with a fast, consistent, and data-driven platform, CafeFlow reduces service time, improves billing accuracy, and gives cafe owners clear visibility into their business.

Keywords: Cafe Management, Order Management, Billing, Inventory, Web Application.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed directly and indirectly to the successful completion of this project. Their guidance, encouragement, and continuous support have been invaluable throughout the entire course of this work.

First and foremost, I would like to express my profound gratitude to the Principal of Phoenix College of Management for the inspiring leadership and academic environment dedicated to learning, research, and innovation.

I would also like to extend my heartfelt appreciation to my project supervisor, **Mr. Saishab Bhattarai**, for the invaluable guidance, constructive feedback, and continuous support throughout the development of this project. Their expertise, patience, and insightful suggestions greatly contributed to the successful completion of this work.

Furthermore, I am deeply thankful to the Department of Information Technology at Phoenix College of Management for providing the necessary academic environment, institutional support, resources, and facilities required for the successful execution of this project.

A special note of thanks goes to my friends and classmates for their constant encouragement, technical discussions, and moral support throughout this journey. Finally, I express my deepest gratitude to my family for their unwavering support, motivation, and encouragement.

CERTIFICATE

This is to certify that the project report entitled **"CafeFlow - A Cafe Management System"** submitted by **Amit Tharu**, **University ID: LC0003001642** to Lincoln University College in partial fulfillment of the requirements for the Bachelor in Information Technology, is a record of original work carried out under my supervision and guidance.

Supervisor Name: Mr. Saishab Bhattarai
Designation: Project Coordinator
Signature:

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

API	Application Programming Interface
BIT	Bachelor in Information Technology
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
POS	Point of Sale
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

Chapter 1:

INTRODUCTION

1.1 Introduction

A cafe handles many small but time-critical tasks every day: taking customer orders, sending them to the kitchen, tracking their progress, producing accurate bills, keeping the menu up to date, and making sure ingredients do not run out. In many small and medium cafes these tasks are still performed manually using notebooks, verbal communication, and spreadsheets. This manual approach is slow, error-prone, and offers no reliable view of sales or stock.

To address these problems, this project proposes CafeFlow, a web-based cafe management system that brings the core operations of a cafe into a single platform. The system allows staff to take and track orders, generate itemised bills, and view a live operational dashboard, while administrators additionally manage the menu, monitor inventory and low-stock alerts, manage staff, and analyse sales performance.

CafeFlow aims to make day-to-day cafe operations faster, more accurate, and data-driven, helping owners deliver better service and make informed business decisions.

1.2 Problem Statement

Despite the growth of the food and beverage sector, many small cafes still operate without a dedicated management system. Orders are recorded on paper and relayed verbally, which causes miscommunication, delays, and mistakes. Bills are calculated by hand, leading to errors and disputes. Stock levels are tracked informally, so ingredients run out unexpectedly. Sales data is rarely recorded in a usable form, leaving owners without insight into revenue, peak hours, or best-selling items. Existing commercial point-of-sale systems are often expensive, complex, or tied to specific hardware, making them impractical for small cafes. There is therefore a need for a simple, affordable, web-based system that unifies ordering, billing, inventory, and reporting in one place.

1.3 Objectives

1. To develop a web-based platform that digitises cafe ordering, billing, inventory, and staff management in a single application.

2. To provide secure, role-based access so that administrators and staff each see only the features relevant to their responsibilities.

3. To generate accurate, real-time sales and inventory analytics that support better business decisions.

1.4 Scope and Limitation

Scope:

1. **Menu Management:** Administrators can create, edit, delete, and toggle the availability of menu items, organised by category.

2. **Order Management:** Staff build orders from the menu and track them through a pending, preparing, and completed workflow.

3. **Billing:** The system generates itemised bills for orders, ready to be printed for the customer.

4. **Inventory Management:** Stock items are tracked with quantities, units, and low-stock thresholds, with automatic low-stock alerts.

5. **Staff Management:** Administrators manage team members and their roles.

6. **Reporting:** The system computes revenue, order counts, best-selling items, and category performance from real order data.

7. **Technology Platform:** The system is web-based, built with TypeScript and SQLite, and is accessible from browsers on desktops, tablets, and smartphones without any installation.

Limitation:

1. **Online Payment:** The current system does not integrate online or card payment gateways; payments are handled offline at the counter.

2. **Single Outlet:** The system is designed for a single cafe outlet and does not yet support multi-branch management.

3. **Internet Dependency:** As a web application, the system requires the server to be reachable over the network to operate.

4. **Kitchen Hardware:** The system does not integrate directly with kitchen display hardware or thermal printers beyond standard browser printing.

1.5 Development Methodology

The CafeFlow system follows the **Agile Methodology**, focusing on iterative development with short cycles of planning, design, implementation, and testing. The project was divided into small increments, each delivering a working feature such as authentication, menu management, order taking, or reporting. This approach allowed continuous refinement and quick adaptation to feedback. Figure 1.1 illustrates the iterative cycle followed throughout development.

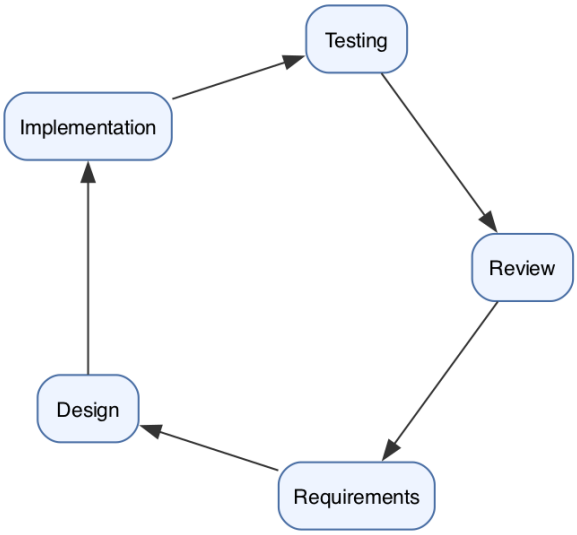

Figure 1.1: Agile / iterative development methodology

1.6 Report Organization

This report is organised into several chapters, each describing a specific area of the project:

Chapter 1: Introduction presents an overview of the project, including the problem statement, objectives, scope, limitations, and methodology.

Chapter 2: Background Study and Literature Review examines the current state of cafe operations and reviews existing systems and technologies relevant to the project.

Chapter 3: System Analysis and Design describes the requirements, feasibility study, and the models and diagrams used to design the system.

Chapter 4: Implementation and Testing explains how the design was translated into a working application and the testing carried out to verify it.

Chapter 5: Conclusion and Future Recommendations summarises the outcomes and proposes directions for future improvement.

Chapter 2:

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Cafes and coffee shops form a large and growing part of the food and beverage industry. Their operations, however, remain highly manual in many small and medium establishments. Orders are commonly taken on paper pads, prices are totalled by hand, and stock is checked by physically inspecting shelves. While these methods are simple, they do not scale well during busy periods and provide no historical record that can be analysed later.

Digital point-of-sale (POS) and restaurant management solutions exist, but they are frequently designed for larger restaurants, require dedicated hardware, or come with subscription costs that are difficult to justify for a small cafe. As a result, many cafe owners continue with manual processes despite their drawbacks.

This project studies these gaps and proposes a lightweight, browser-based alternative that runs on existing devices, requires no installation, and focuses on the specific needs of a cafe: fast ordering, accurate billing, simple inventory control, and clear sales insight.

2.2 Literature Review

This section reviews existing research and systems related to restaurant and cafe automation, point-of-sale technology, web-based management systems, and usability in small-business software. The reviewed works establish the gap that CafeFlow addresses and justify the design decisions taken in this project.

1. Point-of-Sale Systems in Food and Beverage. Ramos and Castro [1] studied point-of-sale (POS) systems in the food and beverage industry and found that, while such systems deliver clear efficiency gains in order taking and billing, their real-world benefit depends strongly on user acceptance and ease of use. Their findings support the central premise of this project, that replacing manual order and billing workflows with a digital system improves speed and accuracy, while also highlighting that the interface must be simple enough for staff to adopt. CafeFlow responds to both points by digitising the order-to-bill workflow behind a clean, role-based interface.

2. Web-Based, Layered System Architecture. Fielding [2] formalised the client–server and layered architectural constraints behind modern web systems, showing that separating user-interface concerns from data-storage concerns improves portability and scalability and allows components to evolve independently. CafeFlow adopts exactly this separation: a browser-based React interface communicates over HTTP with server-side API handlers that sit above a data-access layer, keeping the system maintainable and accessible from any networked device without local installation.

3. Component-Based Front-End Frameworks. A comparative study of front-end frameworks [3] analyses React, Angular, and Vue and concludes that component-based libraries such as React allow complex, interactive interfaces to be built efficiently through reusable components and a predictable, state-driven rendering model, and that framework choice should be a reasoned decision. This is directly relevant to CafeFlow, whose order-builder, dashboard, and reporting screens are highly interactive and benefit from a component-based structure with client-side state management.

4. Lightweight Embedded Databases. Gaffney et al. [4], in a study of SQLite, describe it as a serverless, zero-configuration, fully transactional relational database that is the most widely deployed database engine in the world and is well suited to embedded and single-application use where the overhead of a dedicated database server is unjustified. CafeFlow uses SQLite for exactly this reason: a single cafe outlet needs reliable, ACID-compliant persistence without the cost and administration of a separate database server.

5. Usability and Technology Acceptance. Nielsen [5] derived a refined set of usability heuristics from a factor analysis of real usability problems, providing principles such as visibility of system status, consistency, error prevention, and minimalist design. Complementing this, Davis' Technology Acceptance Model [6] establishes that perceived usefulness and perceived ease of use are the primary determinants of whether users adopt a system. Together these works indicate that adoption of small-business software depends heavily on simplicity and clarity; CafeFlow applies their guidance through a clean, role-based interface, short and consistent workflows, and accessibility from any browser.

In summary, the reviewed literature consistently shows that digitising cafe operations improves efficiency but hinges on user acceptance, that layered web architectures improve scalability and maintainability, that component-based front ends and embedded databases are appropriate technologies for this class of application, and that usability and perceived usefulness are decisive for adoption. CafeFlow combines these established findings into a single, focused cafe management system that is practical for small establishments and addresses the cost and complexity limitations identified in prior work.

Chapter 3:

SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

System analysis identifies what the system must do and the constraints under which it must operate. CafeFlow is analysed as a role-based web application with two types of users — **Administrator** and **Staff** — interacting with seven functional modules: authentication, dashboard, menu, orders, billing, inventory, reporting, and staff management. The analysis below captures the requirements, feasibility, and models that guided the design.

3.2 Requirement Analysis

Functional Requirements:

1. The system shall allow users to log in with an email and password and shall enforce role-based access (admin vs. staff).
2. The system shall allow administrators to create, edit, delete, and toggle availability of menu items.
3. The system shall allow staff to build an order from available menu items, set a table number and optional customer name, and submit it.
4. The system shall track each order through pending, preparing, and completed states.
5. The system shall generate an itemised bill for an order.
6. The system shall allow administrators to manage inventory items and shall raise low-stock alerts.
7. The system shall allow administrators to add and remove staff members and assign roles.
8. The system shall compute and display sales analytics (revenue, orders, top items, category split) from stored order data.

Non-Functional Requirements:

1. **Security:** Passwords shall be stored as bcrypt hashes and sessions managed with secure, HTTP-only cookies.
2. **Usability:** The interface shall be clean and responsive, usable on desktop, tablet, and mobile.
3. **Performance:** Common operations such as loading the menu or submitting an order shall complete within a fraction of a second on a local deployment.
4. **Reliability:** Data shall be persisted to a SQLite database so that it survives restarts.
5. **Maintainability:** The code shall be organised into clear layers (UI, API, data access) to ease future changes.

3.2.1 Feasibility Analysis

Technical Feasibility: The system is built with well-supported, freely available technologies (Next.js, React, TypeScript, SQLite), all of which run on standard hardware. The required skills and tools were available, making the project technically feasible.

Operational Feasibility: The system targets a real, everyday need and offers a simple, role-based interface that cafe staff can use with minimal training, so it is operationally feasible.

Economic Feasibility: The project relies entirely on open-source software and runs on commodity hardware, so development and running costs are minimal, making it economically feasible.

Schedule Feasibility: The project was divided into small Agile increments with clear milestones, allowing it to be completed within the available academic timeframe.

3.2.2 System Modelling

The system is modelled using standard UML and process diagrams. The **order processing flow** in Figure 3.1 shows the lifecycle of an order from creation to billing.

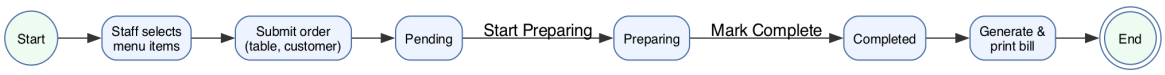

Figure 3.1: Order processing flow of CafeFlow

3.3 System Design

The system design translates the requirements and models into concrete structures: the user interface, the data model, and the runtime architecture.

3.3.1 Interface Design (UI/UX)

The interface uses a clean, role-based layout with a fixed sidebar for navigation and a content area for each module. Figure 3.2 shows the login screen and Figure 3.3 shows the administrator dashboard with live statistics and charts.

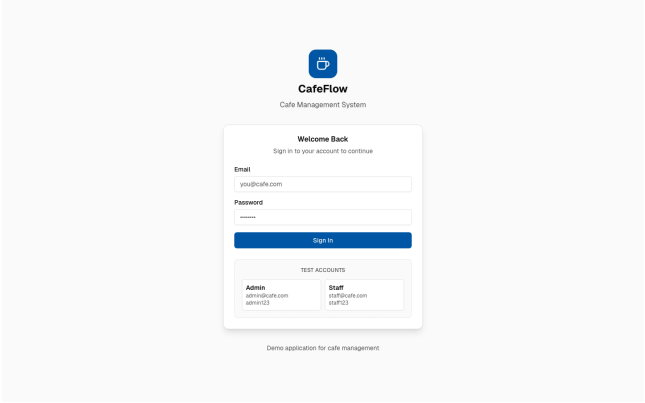

Figure 3.2: Login interface of CafeFlow

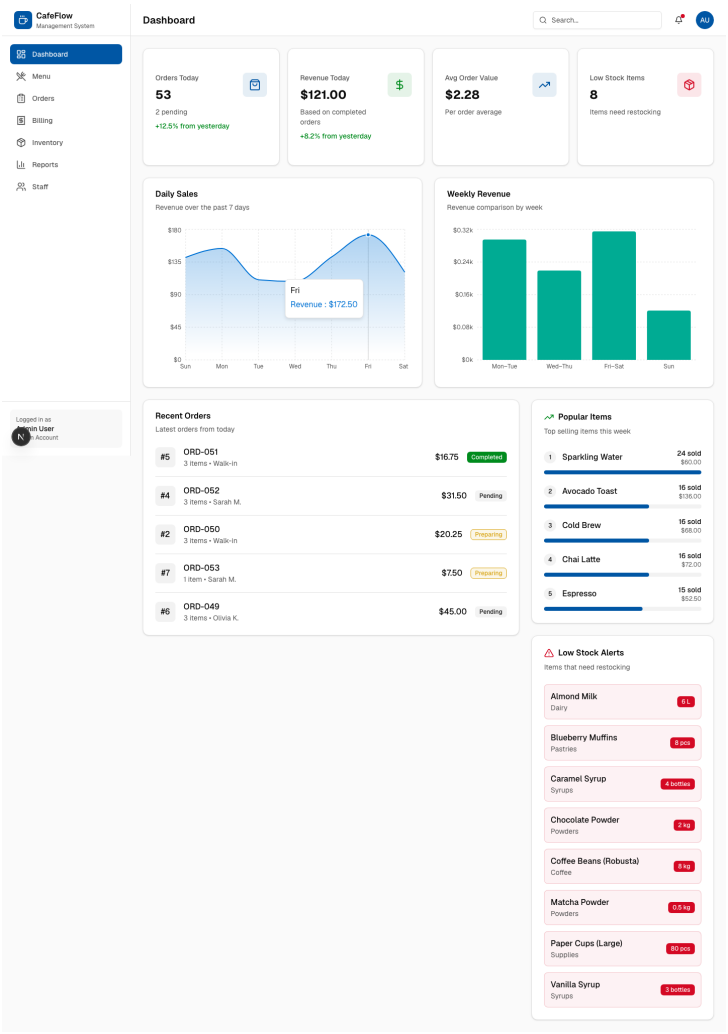

Figure 3.3: Administrator dashboard of CafeFlow

3.3.2 Refinement of Classes and Object (Data Model)

The data is stored in a relational SQLite schema. The **entity relationship diagram** in Figure 3.4 shows the main tables and their relationships: `users` and `sessions` support authentication; `menu_items` and `inventory_items` hold reference data; and `orders` and `order_items` record transactions, with each order line storing a snapshot of the item name and price so that historical bills remain correct even if the menu later changes.

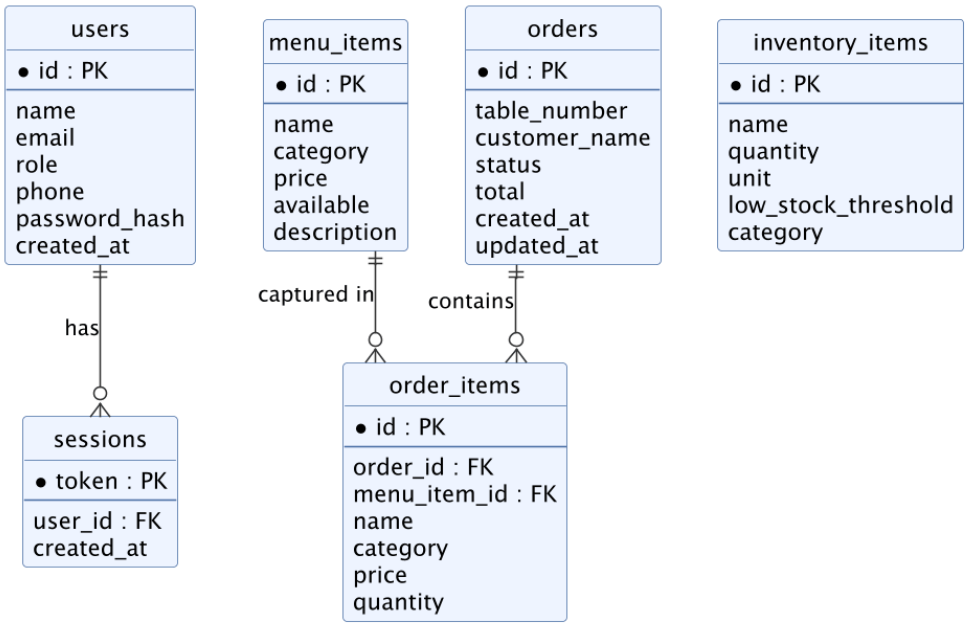

Figure 3.4: Entity relationship diagram of CafeFlow

3.3.3 Component Diagram

The system follows a layered architecture. The browser runs the React user interface and client-side state stores, which communicate over HTTP with server-side API route handlers. The handlers apply authentication and authorisation, then use a data-access layer to read and write the SQLite database. Figure 3.5 shows these components.

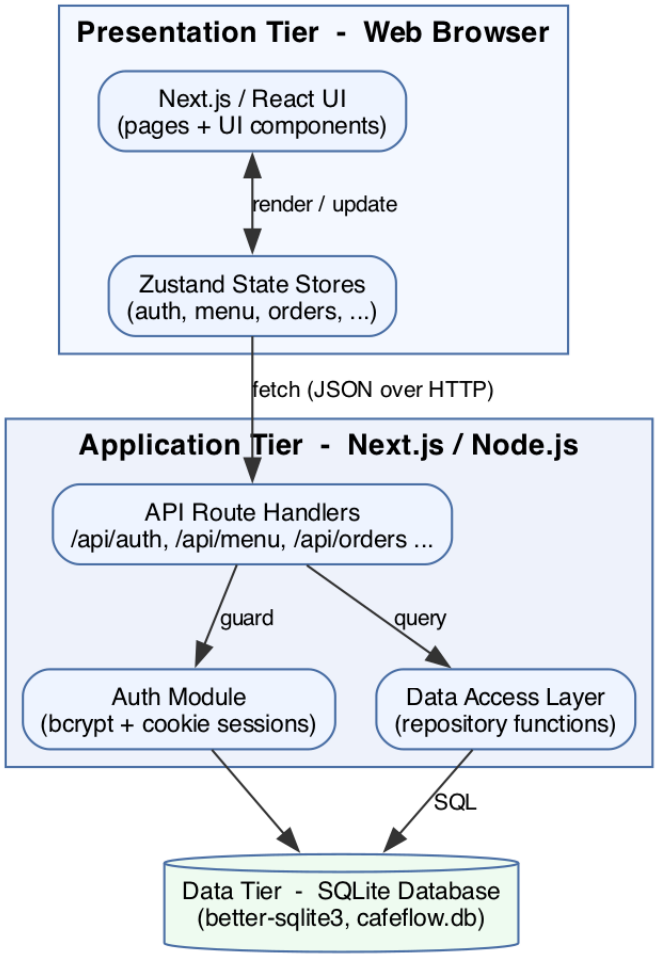

Figure 3.5: Component / architecture diagram of CafeFlow

3.3.4 Deployment Diagram

At runtime the application is deployed as a single Node.js server that serves the Next.js application and exposes the API, backed by a SQLite database file on the same host. Clients access the system through a web browser. Figure 3.6 shows the deployment.

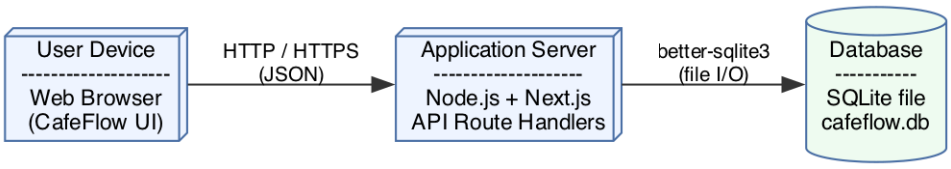

Figure 3.6: Deployment diagram of CafeFlow

Chapter 4:

IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools Used

The system was implemented using the following tools and technologies:

- **Next.js & React:** Full-stack framework and UI library for building the application and its API routes.
- **TypeScript:** Typed JavaScript for safer, more maintainable code.
- **Tailwind CSS & shadcn/ui:** Utility-first styling and accessible UI components.
- **Zustand:** Lightweight client-side state management.
- **SQLite (better-sqlite3):** Embedded relational database for persistence.
- **bcrypt:** Password hashing for secure authentication.
- **Recharts:** Charting library for the dashboard and reports.
- **Playwright:** End-to-end testing framework.
- **Visual Studio Code & Git/GitHub:** Development environment and version control.

4.1.2 Implementation Details of Modules

Authentication: Users sign in with an email and password. Passwords are verified against bcrypt hashes stored in the `users` table; on success a session token is stored in the `sessions` table and set as a secure HTTP-only cookie. Every protected API endpoint validates the session, and administrator-only actions additionally check the user's role.

Menu: Administrators manage menu items through create, edit, delete, and availability-toggle operations, with search, category filtering, and table/card views.

Orders: Staff build an order by selecting available items, adjusting quantities, and entering a table number and optional customer name. Submitting an order stores it as

`pending`; it is then advanced to `preparing` and `completed`. Each order line stores a snapshot of the item name and price.

Billing: Completed and in-progress orders can be selected to produce an itemised bill showing the cafe header, order details, line items, and total, ready for printing.

Inventory: Stock items are tracked with quantity, unit, category, and a low-stock threshold. Items at or below their threshold are flagged, and the dashboard shows low-stock alerts.

Staff: Administrators add and remove team members and assign the admin or staff role.

Reports: Revenue, order counts, average order value, best-selling items, and category performance are computed from stored order data and presented as summary cards and charts.

4.2 Testing

The system was tested using automated end-to-end tests written with Playwright, which drive a real browser through the application exactly as a user would. The tests cover every module and include both unit-level checks of individual functions and system-level checks of complete workflows. In total, 25 automated tests were executed, all of which passed.

4.2.1 Test Cases for Unit Testing

Table 4.1 lists representative unit test cases that verify individual functions and behaviours.

Table 4.1: Test Cases for Unit Testing

ID	Test Case	Expected Result	Result
U01	Login with valid admin credentials	User is authenticated and redirected to dashboard	Pass
U02	Login with an invalid password	Error message “Invalid email or password” is shown	Pass
U03	Access a protected API without a session	Request is rejected with 401 Unauthorized	Pass
U04	Add a menu item with valid data	Item is created and appears in the menu list	Pass

ID	Test Case	Expected Result	Result
U05	Toggle a menu item's availability	Availability switches and a confirmation is shown	Pass
U06	Increase an item's quantity in the order cart	Quantity increments and the total updates	Pass
U07	Add an inventory item	Item is stored and listed in inventory	Pass
U08	Add a staff member with a duplicate email	Request is rejected with a conflict error	Pass
U09	Compute order total from line items	Total equals the sum of price × quantity	Pass
U10	Reports summary from stored orders	Revenue and order counts are calculated correctly	Pass

4.2.2 Test Cases for System Testing

Table 4.2 lists system test cases that verify complete, multi-step workflows.

Table 4.2: Test Cases for System Testing

ID	Test Case	Expected Result	Result
S01	Staff signs in and is restricted to permitted sections	Admin-only links (Menu, Reports, Staff) are hidden	Pass
S02	Staff visits an admin-only route directly	User is redirected back to the dashboard	Pass
S03	Full menu lifecycle: add, toggle, edit, delete	Each operation succeeds and the list updates	Pass
S04	Build and submit an order	Order is created and appears under the Pending tab	Pass
S05	Advance an order pending → preparing → completed	Status updates at each step with confirmation	Pass
S06	Generate and print a bill for an order	Itemised bill is shown and print is confirmed	Pass
S07	Add, edit, and delete an inventory item	Inventory list reflects each change	Pass

ID	Test Case	Expected Result	Result
S08	Add a staff member then remove with confirmation	Member is added, then removed after confirming	Pass
S09	View reports and switch analytics tabs	Summary cards and charts render for each tab	Pass
S10	Log out	Session ends and the user returns to the login page	Pass

4.2.3 Result Analysis

All 25 automated test cases passed, covering authentication and role-based access, menu, order, billing, inventory, staff, and reporting workflows. The results confirm that the system meets its functional requirements and behaves correctly across complete user journeys. The automated suite also serves as a regression guard, allowing future changes to be validated quickly. No critical defects remained at the end of testing; minor issues found during development (such as session handling and state updates) were fixed and re-verified.

Chapter 5:

CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

CafeFlow successfully delivers a complete, web-based cafe management system that unifies menu management, order taking, billing, inventory tracking, staff management, and sales reporting in a single role-based application. By replacing manual, paper-based processes with a fast and consistent digital workflow, the system reduces service time, improves billing accuracy, and provides cafe owners with clear, real-time insight into their operations. The project met all of its stated objectives and was validated by a comprehensive automated test suite.

5.2 Lesson Learnt / Outcome

The project provided valuable experience in designing and building a full-stack web application end to end. Key lessons included structuring a codebase into clear layers (interface, API, and data access), securing an application with hashed passwords and session-based authentication, modelling relational data and preserving historical accuracy through snapshots, and validating an application thoroughly with automated end-to-end tests. The work also reinforced the value of an iterative, Agile approach in delivering a working product incrementally.

5.3 Future Recommendations

While CafeFlow meets its current objectives, several enhancements are recommended for the future:

1. **Online Payments:** Integrate digital wallets and card payment gateways for cashless transactions.
2. **Multi-Outlet Support:** Extend the system to manage multiple cafe branches from a single account.
3. **Kitchen Display & Printing:** Add dedicated kitchen display screens and direct thermal-printer support.

4. **Customer-Facing Ordering:** Provide QR-code table ordering so customers can place orders directly.

5. **Advanced Analytics:** Add demand forecasting and automatic stock reorder suggestions.

REFERENCES

[1] Y. Ramos and A. Castro, "Point-of-sales systems in food and beverage industry: Efficient technology and its user acceptance," *Journal of Information Sciences and Computing Technologies*, vol. 6, no. 1, pp. 582–589, 2017.

[2] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000. [Online]. Available: https://roy.gbiv.com/pubs/dissertation/top.htm

[3] "Analysis and comparison of different frontend frameworks," in *Applications and Techniques in Information Security (ATIS 2022)*, ser. Communications in Computer and Information Science. Singapore: Springer Nature, 2023, pp. 243–257, doi: 10.1007/978-981-99-2264-2_19.

[4] K. P. Gaffney, M. Prammer, L. Brasfield, D. R. Hipp, D. Kennedy, and J. M. Patel, "SQLite: Past, present, and future," *Proc. VLDB Endowment*, vol. 15, no. 12, pp. 3535–3547, 2022, doi: 10.14778/3554821.3554842.

[5] J. Nielsen, "Enhancing the explanatory power of usability heuristics," in *Proc. SIGCHI Conf. on Human Factors in Computing Systems (CHI '94)*, Boston, MA, USA, 1994, pp. 152–158, doi: 10.1145/191666.191729.

[6] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989, doi: 10.2307/249008.

[7] Vercel, "Next.js Documentation," [Online]. Available: https://nextjs.org/docs

[8] Meta, "React Documentation," [Online]. Available: https://react.dev

[9] Microsoft, "TypeScript Documentation," [Online]. Available: https://www.typescriptlang.org/docs

[10] SQLite Consortium, "SQLite Documentation," [Online]. Available: https://www.sqlite.org/docs.html

[11] J. Wiegley, "better-sqlite3," [Online]. Available: https://github.com/WiseLibs/better-sqlite3

[12] Tailwind Labs, "Tailwind CSS Documentation," [Online]. Available: https://tailwindcss.com/docs

[13] pmndrs, “Zustand,” [Online]. Available: https://github.com/pmndrs/zustand

[14] Microsoft, “Playwright Documentation,” [Online]. Available: https://playwright.dev

[15] D. Mazières, “bcrypt password hashing,” [Online]. Available: https://en.wikipedia.org/wiki/Bcrypt

[16] Recharts Group, “Recharts,” [Online]. Available: https://recharts.org

APPENDICES

The following screenshots show the main modules of the working CafeFlow application.

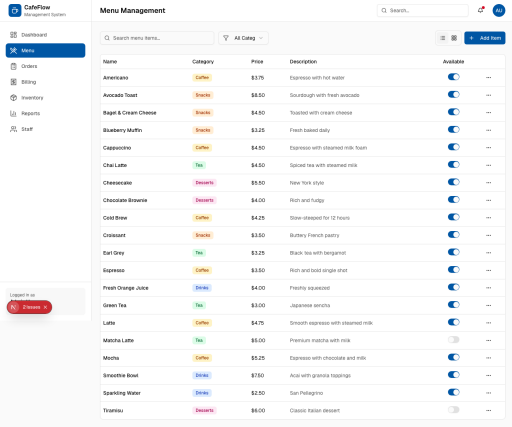

Figure 5.1: Menu management

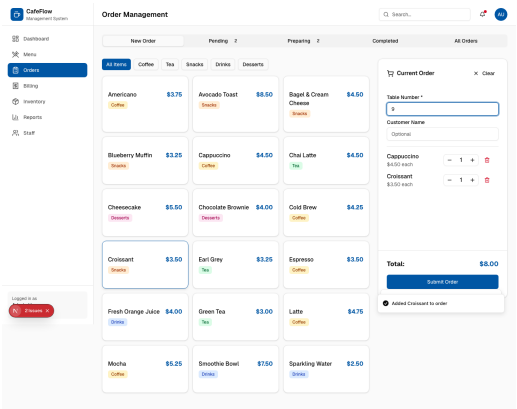

Figure 5.2: Order builder (new order)

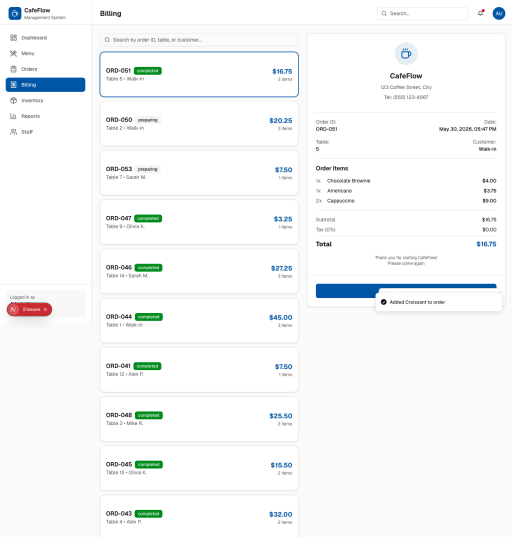

Figure 5.3: Billing and itemised bill preview

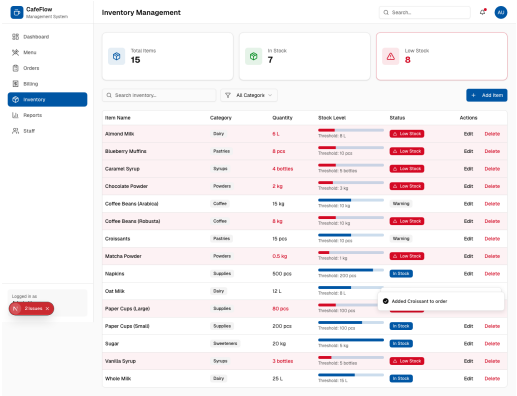

Figure 5.4: Inventory management

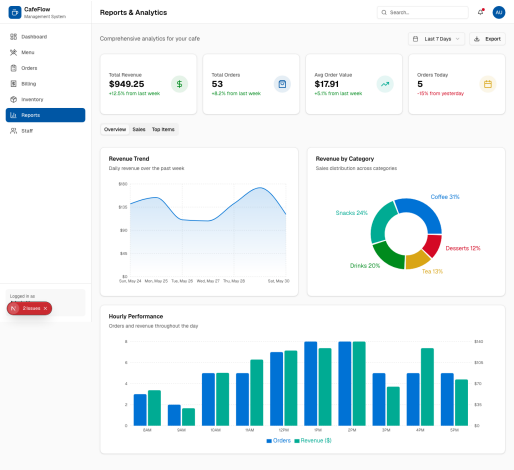

Figure 5.5: Reports and analytics

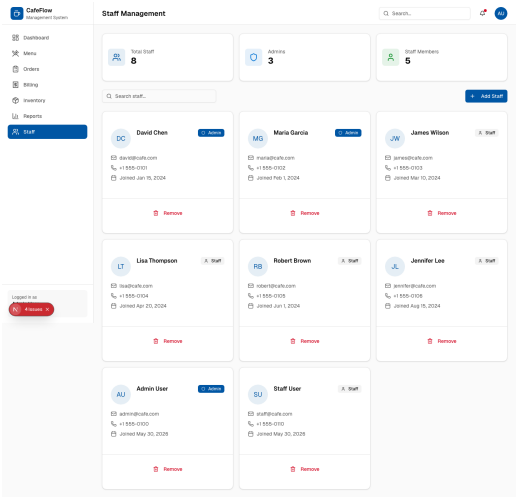

Figure 5.6: Staff management

Project Log Book Entry Sheet

CafeFlow – A Cafe Management System | Amit Tharu

Meeting No.: 01 **Date: 23/11**

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Introduction to final year project requirements
2. Explanation of project phases and deadlines
3. Overview of documentation chapters (Introduction, Literature Review, etc.)
4. IEEE format, abstract writing, and reference guidelines

Achievements:

1. Understood complete project roadmap
2. Received project guideline document

Problems (if any):

None

Task for next meeting:

Brainstorm project ideas

Meeting No.: 02 **Date: 30/11**

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Presented the project ideas
2. Finalized "CafeFlow – A Cafe Management System"
3. Discussed problem statement, objectives and significance

Achievements:

1. Project title approved
2. Problem statement drafted

Problems (if any):

None

Task for next meeting:

Write Introduction chapter; perform feasibility study

Meeting No.: 03 **Date: 14/12**

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. How to write literature review
2. Citation and reference in IEEE format
3. Use of Google Scholar and research papers

Achievements:

1. Learned IEEE citation style

2. Collected research papers for literature review

Problems (if any):

None

Task for next meeting:

Complete literature review draft; prepare reference list

Meeting No.: 04 **Date: 28/12**

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Reviewed Introduction and Literature Review chapters
2. Finalized functional and non-functional requirements
3. Feasibility study (technical, operational, economic, schedule)

Achievements:

1. Chapters 1 and 2 completed
2. Feasibility confirmed

Problems (if any):

None

Task for next meeting:

Start system design (UML diagrams); decide technology stack

Meeting No.: 05 **Date: 11/01**

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. System design chapter writing
2. UML diagrams (ER, Component, Deployment, Process Flow)
3. Project proposal format and submission deadline
4. Mid defense preparation guidelines

Achievements:

1. Learned to create diagrams using Draw.io / Graphviz
2. Understood proposal structure
3. Started working on ER and Component diagrams

Problems (if any):

None

Task for next meeting:

Complete all UML diagrams; submit project proposal

Meeting No.: 06 **Date: 25/01**

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Reviewed ER and Component diagrams

2. Technology stack confirmation (Next.js, TypeScript, SQLite)

Achievements:

1. All UML diagrams approved by supervisor
2. Development environment setup (VS Code, GitHub, Node.js)

Problems (if any):

1. Confusion in showing ER relationships (one-to-many vs many-to-many); supervisor helped correct them.

Task for next meeting:

Start frontend development (login and dashboard); set up project with TypeScript

---

Meeting No.: 07 **Date: 01/02**

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Authentication and dashboard implementation progress
2. Role-based access (Admin vs Staff)
3. Menu and order management modules

Achievements:

1. Email/password authentication working
2. Role-based dashboards for Admin and Staff
3. Database connected successfully (SQLite)

Problems (if any):

1. Session management not working initially. Fixed using secure HTTP-only cookies.

Task for next meeting:

Build order and billing modules; prepare for mid defense

Meeting No.: 08 **Date: 08/03**

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Presentation to supervisor (15 minutes)
2. Live demo of working features (login, menu, orders, billing)
3. Questions and feedback

Achievements:

1. Mid defense cleared successfully
2. Supervisor appreciated the concept and working prototype
3. Received constructive feedback

Problems (if any):

1. Supervisor said the user interface could be more polished.
2. Examiners pointed out that test cases were not sufficient.

Task for next meeting:

Improve UI design; add more test cases; complete remaining report chapters

Meeting No.: 09 **Date: 29/03**

Start Time: 9:00 AM Finish Time: 10:00 AM

Discussion Topics:

1. Final report review (all chapters)
2. Abstract writing and conclusion
3. Future recommendations
4. Final defense presentation and viva preparation

Achievements:

1. Final report draft completed in LaTeX
2. All diagrams inserted properly
3. 25 automated end-to-end tests passing (Playwright)
4. Presentation slides prepared

Problems (if any):

1. Figure placement in LaTeX was tricky. Solved using the [H] option with the float package.
2. Difficulty formatting test case tables. Used longtable to fix.

Task for next meeting:

Submit final report; prepare for viva questions

Meeting No.: 10 **Date: 17/05**

Start Time: 10:00 AM Finish Time: 11:00 AM

Discussion Topics:

1. Final defense schedule and date confirmation
2. Rules and regulations for final defense
3. Presentation duration and discipline
4. Documents to bring on defense day; backup slides and live demo guidelines

Achievements:

1. Understood all rules and regulations for final defense
2. Prepared checklist for defense day

Problems (if any):

None

Task for next meeting:

Complete final report submission

Student's Name: **Supervisor Signature**

Amit Tharu